BTK AKADEMİ

Her an,
her yerde
öğren !

Yapay Zeka ve Üretken Modeller Atölyesi - 2

Dr.Öğr.Üyesi Furkan Göz

Makine Öğrenmesine Giriş

- Makine öğrenmesi sistemlerin açıkça programlanmaksızın veri üzerinden öğrenmesini sağlayan bir yapay zeka alt alanıdır.
- **Amaç:** Girdilerle çıktılar arasındaki örüntüyü öğrenip yeni ve daha önce görülmemiş verilere uygun tahminler üretmektir.
- **Uygulama Alanları:** E-posta filtreleme, kredi risk skorlama, sağlık teşhis sistemleri, görüntü tanıma, öneri motorları.
- Makine öğrenmesi; **istatistik**, **veri bilimi** ve **bilgisayar bilimlerinin** kesişim noktasında yer alır.

Neden Makine Öğrenmesi?

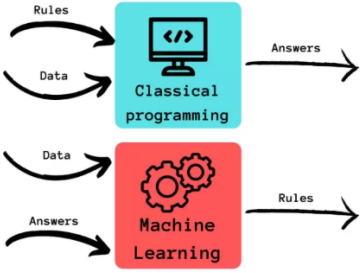
- Geleneksel programlama: *”Kuralları insan yazar, sistem çıktıyı üretir.”*
- Makine öğrenmesi: *”Veri ve sonuç verilir, sistem kuralları kendi öğrenir.”*
- Kural yazılamayacak kadar karmaşık veya dinamik sistemlerde (örneğin görsel tanıma, doğal dil işleme) insan programcı yetersiz kalabilir.
- Büyük veri çağında devasa veri yığınlarından anlamlı sonuçlar çıkarabilen otomatize sistemlere ihtiyaç vardır.
- Karar destek sistemleri otomasyon ve kişiselleştirme gibi alanlarda başarısı kanıtlanmıştır.

Not: Makine öğrenmesi kural odaklı düşünmek yerine **veri odaklı (data-driven) düşünmeyi** zorunlu kılar.

Klasik Programlama vs. Makine Öğrenmesi

- **Klasik Programlama**
  - Kurallar (kod) ve veriler bilgisayara verilir.
  - Bilgisayar bu kuralları uygular ve sonuç üretir.

- **Makine Öğrenmesi**
  - Veriler ve cevaplar (etiketli veri) sağlanır.
  - Bilgisayar kuralları (modeli) kendi öğrenir.



Makine Öğrenmesinde Öğrenme Türleri

- Makine öğrenmesi algoritmaları verinin etiketli olup olmamasına ve öğrenme stratejisine göre sınıflara ayrılır:

  1. Gözetimli Öğrenme (Supervised Learning)

  - Her veri örneği bir girdi (X) ve bilinen, doğru bir çıktı/etiket (y) içerir.
  - **Amaç:** Girdi ile çıktı arasındaki matematiksel ilişkiyi öğrenerek gelecekteki yeni verilere doğru tahminler yapabilmektir.
  - **Alt Dalları:** Sınıflandırma (Classification - örn. spam tespiti) ve Regresyon (Regression - örn. ev fiyatı tahmini).

Makine Öğrenmesinde Öğrenme Türleri

2. Gözetimsiz Öğrenme (Unsupervised Learning)

- Sadece giriş verileri (X) mevcuttur, hedeflenen bir çıkış etiketi (y) yoktur.
- **Amaç:** Veri seti içindeki gizli yapıları, benzerlikleri, grupları veya dağılımları keşfetmektir.
- **Alt Dalları:** Kümeleme (Clustering - örn. müşteri segmentasyonu) ve Boyut İndirgeme (Dimensionality Reduction - örn. PCA).

Makine Öğrenmesinde Öğrenme Türleri

- **3. Pekiştirmeli Öğrenme (Reinforcement Learning)**
  - Bir ajanın (agent), çevreyle (environment) etkileşime girerek deneme yanılma yoluyla öğrenmesidir.
  - **Amaç:** Ödülü (reward) maksimize cezayı (penalty) minimize edecek en iyi davranış stratejisini (policy) bulmaktır.
  - **Kullanım Alanları:** Otonom araçlar, oyun yapay zekaları (Satranç, Go), robotik navigasyon.

- **4. Yarı Gözetimli Öğrenme (Semi-Supervised Learning)**
  - Az sayıda etiketli veri ve çok sayıda etiketsiz verinin birlikte kullanıldığı yaklaşımdır. Veri etiketleme maliyetini düşürmek için tercih edilir.

Veri ile Model Arasındaki İlişki

- Makine öğrenmesinde model verilerdeki örüntüyü (pattern) ve kuralları matematiksel olarak yakalamaya çalışır.
- Her veri noktası: **Özellikler (Features, X)** $\rightarrow$ **Çıktı (Etiket, y)**
- Model örneklerden yola çıkarak hiç görmediği veriler üzerinde **genelleme (generalization)** yapmayı öğrenir.
- Model = Matematiksel Fonksiyon + Öğrenilen Parametreler (Ağırlıklar)
- **Temsili form:** $\hat{y} = f(X)$
- Modelin başarısı problemin **doğru veri temsiline** dönüştürülmesine bağlıdır.

Örnek: Bir hastanın yaş, tansiyon ve nabız bilgisi (X) $\rightarrow$ Kalp krizi riski (y: Evet/Hayır)

Modelleme Süreci ve Bileşenleri (ML Lifecycle)

- Bir makine öğrenmesi projesi döngüsel bir süreçtir:
  1. **Veri Toplama:** Gerçek dünyadan veya veri tabanlarından ham örneklere ulaşılması.
  2. **Veri Hazırlama (Preprocessing):** Temizleme, dönüştürme ve eksikleri giderme.
  3. **Özellik Seçimi (Feature Engineering):** Modele en çok katkı sağlayacak değişkenlerin üretilmesi ve seçilmesi.
  4. **Model Seçimi:** Problem türüne (Regresyon/Sınıflandırma) en uygun mimarinin belirlenmesi.
  5. **Model Eğitimi (Training):** Verideki matematiksel örüntülerin algoritmaya öğretilmesi.
  6. **Değerlendirme (Evaluation):** Test verisi ile modelin genelleme performansının ölçülmesi.
  7. **İyileştirme (Tuning):** Hata analizi, hiperparametre ayarı ve gerekirse yeni veri toplanması.

Veri Hazırlama (Data Preprocessing) Adımları

- **Garbage In, Garbage Out (Çöp girerse çöp çıkar).** İyi hazırlanmamış bir veri seti en gelişmiş derin öğrenme modelini bile başarısız yapar.
- **Temel Adımlar**
  - **1. Veri Ayrıştırma:** Girdi (Bağımsız değişkenler, X) ve Çıktı (Bağımlı değişken, y) ayrımı.
  - **2. Eksik Veri Yönetimi:** Eksik satırları silme, istatistiksel doldurma (Imputation) veya tahmin etme.
  - **3. Özellik Ölçekleme (Scaling):** Değişkenlerin matematiksel olarak aynı ağırlığa/aralığa getirilmesi (Min-Max, Z-skor).
  - **4. Etiket Kodlama (Encoding):** Metinsel ve kategorik verilerin modelin anlayacağı vektörlere/sayılara dönüştürülmesi.
  - **5. Veri Bölme (Train/Test Split):** Modelin ezberleyip ezberlemediğini (Overfitting) kontrol etmek için verinin izole edilmesi.

Eksik Verilerle Başa Çıkma (Missing Data Imputation)

- Gerçek veri setlerinde sensör arızaları, kullanıcı ihmalleri veya sistemsel hatalar sebebiyle eksik değerler (NaN/Null) kaçınılmazdır.
-
  - **Silme (Dropping):** Eksik satırları veya tamamen eksik sütunları çıkarma.
    - *Avantaj:* Uygulaması kolaydır, veriyi manipüle etmez.
    - *Dezavantaj:* Veri kaybına ve modelin öğreneceği örnek sayısında azalmaya yol açar.
  - **Doldurma (Imputation)**
    - *Sayısal değişkenler:* Ortalama (aykırı değer yoksa), medyan (aykırı değer varsa), mod.
    - *Kategorik değişkenler:* En sık tekrar eden sınıf (mode) veya ”Bilinmiyor” adında yeni bir kategori oluşturma.
    - *Model Tabanlı Tahmin:* K-NN veya Regresyon ile eksik değerin diğer özelliklere bakılarak tahmin edilmesi.

Eksik Veri: Silmek mi, Doldurmak mı?

Duruma ve veri yapısına göre karar verilmelidir:

- **1. Veri seti küçükse & çok eksik varsa:**
  - *Örnek:* 100 gözlem var, 30'unda yaş bilgisi eksik.
  - Silme → Veri kaybı çok büyük → **Doldurma tercih edilir.**
- **2. Eksik veri model için önemsiz bir değişkendeyse:**
  - *Örnek:* Kredi onay modelinde "ikinci e-posta adresi" eksikliği.
  - Sütunu tamamen **silmek en uygunudur.**
- **3. Eksik veri tahmin gücü yüksek, kritik bir değişkendeyse:**
  - *Örnek:* Diyabet tahmininde "Kan şekeri" eksikliği.
  - Ortalama ile doldurmak veriyi bozar; **model tabanlı gelişmiş doldurma** önerilir.
- **4. Eksiklik oranı çok düşükse (%1 - %2):**
  - O satırları **silmek** istatistiksel olarak veri setine zarar vermez.

Eksik Veriler – Python Örneği

```python
import pandas as pd
from sklearn.impute import SimpleImputer

df = pd.read_csv("veri.csv")
print("Eksik veri sayisi:\n", df.isnull().sum())

# 1. Yontem: Pandas ile hizli doldurma
df["yas"] = df["yas"].fillna(df["yas"].median())

# 2. Yontem: Scikit-Learn
imputer = SimpleImputer(strategy="median")
df[["maas"]] = imputer.fit_transform(df[["maas"]])
```

Özellik Ölçekleme (Feature Scaling) ve Veri Sızıntısı

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Z-score standardizasyonu (Ortalama 0, Varyans 1)
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

- **Veri Sızıntısı (Data Leakage):** `fit` işlemi sütunun ortalamasını ve maksimum değerini öğrenir.

Etiket Kodlama (Encoding) Nedir?

- Makine öğrenmesi algoritmalarının temelinde türev, matris çarpımı ve mesafe hesaplamaları yatar. Bu nedenle modeller metinsel (kategorik) verilerle doğrudan çalışamaz (”Mavi” veya ”Ankara” gibi).
- Kategorik verilerin matematiksel olarak anlamlı sayısal formlara dönüştürülmesine **etiket kodlama (encoding)** denir.
- Doğru kodlama yöntemi seçilmezse model yanlış varsayımlarda bulunur ve başarısız olur.

Label Encoding (Sıralı Kodlama)

- Her benzersiz kategoriye $0, 1, 2...$ şeklinde ardışık tam sayılar atanır.
- **Doğru Kullanım (Ordinal Veri):** Kategoriler arasında doğal bir büyüklük/sıra ilişkisi varsa uygundur.
  - Örn: Eğitim Durumu (İlkokul=0, Lise=1, Üniversite=2). Model $2 > 1 > 0$ mantığını doğru kurar.
- **Yanlış Kullanım (Nominal Veri):** Sırasız kategorilerde kullanıldığında algoritmayı yanıltır.
  - Örn: `Kedi=0`, `Köpek=1`, `Kuş=2`.

Label Encoding – Python Örneği

```python
from sklearn.preprocessing import LabelEncoder

df = pd.DataFrame({"egitim": ["Lise", "Ilkokul", "Universite", "
    Lise"]})

encoder = LabelEncoder()
df["egitim_kodlu"] = encoder.fit_transform(df["egitim"])
print(df)

#         egitim  egitim_kodlu
# 0         Lise             1
# 1      Ilkokul             0
# 2   Universite             2
# 3         Lise             1
```

One-Hot Encoding

- Sırası olmayan (Nominal) veriler için kullanılır. Her kategori için yeni bir sütun açılır ve ilgili satıra 1, diğerlerine 0 yazılır.
- **Avantaj:** Sayılar arasında matematiksel bir büyüklük küçüklük ilişkisi kurulmasını engeller.
- **Dezavantaj (Boyut Laneti):** Kategori sayısı çok yüksekse (Posta kodları) binlerce yeni sütun oluşur.

One-Hot Encoding – Python Örneği

```
import pandas as pd

df = pd.DataFrame({"renk": ["Kirmizi", "Mavi", "Yesil", "Mavi"
    ]})

# drop_first=True
# Eger Kirmizi ve Mavi 0 ise model otomatik olarak Yesil
    oldugunu anlar.
onehot_df = pd.get_dummies(df, columns=["renk"], drop_first=True
    , dtype=int)
print(onehot_df)

#    renk_Mavi  renk_Yesil
# 0          0           0    <-- İ(kisi de 0 ise renk ııııKrmzdr)
# 1          1           0
# 2          0           1
# 3          1           0
```

Etiket Kodlama

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

df = pd.DataFrame({"renk": ["kirmizi", "mavi", "yesil", "kirmizi
    "]})

# 1. Label Encoding (Agac modelleri veya sirali veriler icin)
le = LabelEncoder()
df["renk_label"] = le.fit_transform(df["renk"])

# 2. One-Hot Encoding (Uzaklik tabanli modeller ve sirasiz
    veriler icin)
df_ohe = pd.get_dummies(df["renk"], prefix="renk", drop_first=
    True, dtype=int)
```

Model Doğrulama: Eğitim ve Test Bölünmesi (Train/Test Split)

- Modelin ezberlemediğini (genelleme yeteneğini) kanıtlamak için veri seti parçalara ayrılır.
- **Eğitim Seti (Train):** Modelin matematiksel örüntüleri ve algoritmik ağırlıkları öğrendiği veri kümesidir.
- **Test Seti (Test):** Modelin daha önce hiç görmediği, gerçek dünya performansını simüle eden değerlendirme kümesidir.

- Yaygın bölünme oranları: `80/20` veya `70/30`.
- **Alternatif Yaklaşımlar:** Çapraz Doğrulama (Cross-Validation) veya Eğitim/Doğrulama/Test (Train/Validation/Test) üçlü mimarisi.

Kural: Test verisi eğitim aşamasında ASLA kullanılmaz.

Veri Bölme Stratejileri ve Riskler

- **Temsil Gücü (Representativeness):** Eğitim ve test setleri, ana veri setinin istatistiksel dağılımını yansıtmalıdır. Sadece belirli bir grubun teste gitmesi başarıyı yanıltır.
- **Dengesiz Sınıflar (Class Imbalance):** Nadir görülen sınıfların test setinde hiç yer almaması riskini önlemek için **Stratified Split** kullanılabilir.
- **Zaman Serisi (Time-Series):** Veri zamana bağlıysa rastgele bölme yapılamaz. Geleceği tahmin etmek için geçmiş veriler eğitime gelecekteki veriler teste ayrılmalıdır (Out-of-Time Validation).

Sınıf Dağılımını Korumak: `stratify=y`

```
# Hedef (y) dagilimi: %90 Sinif 0, %10 Sinif 1
y = [0]*90 + [1]*10

# Yanlis Yaklasim: Rastgele bolme (Test setine hic 1
    gelmeyebilir!)
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2)

# Stratify ile orantili bolme
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
```

Veri Setini İşleme Hazırlama (Python Uygulaması)

```python
from sklearn.model_selection import train_test_split

X = df.drop("hedef", axis=1)
y = df["hedef"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
```

Makine Öğrenmesi: Fit ve Predict

- Scikit-Learn algoritmaları iki temel metot üzerinden çalışır:
- **1. Öğrenme Aşaması (`fit`)**
  - Model, `X_train` (özellikler) ile `y_train` (cevaplar) arasındaki matematiksel fonksiyonu optimize eder. Ağırlıklar güncellenir.
- **2. Tahmin Aşaması (`predict`)**
  - Model, öğrendiği kuralları hiç cevaplarını (y) bilmediği `X_test` verisine uygulayarak kendi tahminlerini (`y_pred`) üretir.
- **Değerlendirme:** Nihai ölçüm, modelin ürettiği `y_pred` ile gerçek cevaplar olan `y_test`'in karşılaştırılmasıyla yapılır.

Makine Öğrenmesi Algoritmaları

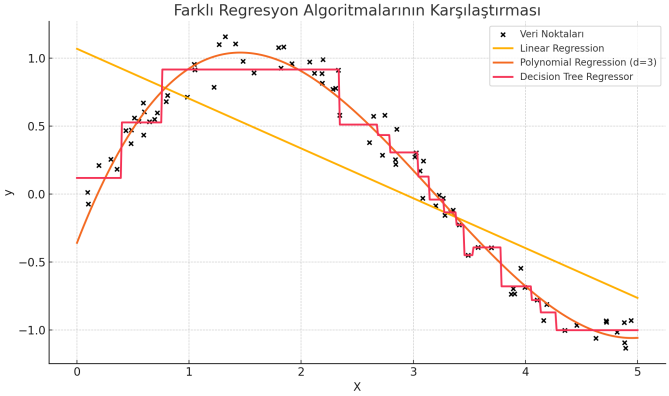
- **No Free Lunch (Bedava Öğle Yemeği Yok)** teoremine göre; her veri setinde en iyi çalışan bir algoritma yoktur.
- **1. Regresyon:** Sürekli ve sayısal değer tahmini (Örn: Maaş, Ev fiyatı, Sıcaklık).
- **2. Sınıflandırma:** Veriyi ayrık kategorilere atama (Örn: E-posta spam mi?).
- **3. Kümeleme (Gözetimsiz):** Benzer verileri etiket olmadan gruplayarak örüntü keşfi (Örn: Müşteri segmentasyonu).

Kural: Verinin boyutuna, tipine ve problemin doğasına göre uygun algoritma seçilmeli ve birden fazla model birbiriyle yarıştırılmalıdır.

Regresyon Algoritmaları (Sürekli Değer Tahmini)

- **Doğrusal Regresyon (Linear Regression)**
  - En temel regresyon modelidir. Özellikler ile hedef arasındaki ilişkiyi doğrusal bir çizgi ($y = wX + b$) ile çizer.
  - *Örnek:* Evin m²'sine ve oda sayısına göre fiyat tahmini.
- **Polinomsal Regresyon (Polynomial Regression)**
  - Veri doğrusal bir trend izlemiyorsa, eğrisel (non-linear) ilişkilere uyum sağlamak için X^2, X^3 gibi dereceler kullanılır.
- **Karar Ağacı Regresyonu (Decision Tree Regressor)**
  - Veri setini küçük parçalara (bölmelere) ayırarak her bölme için ortalama bir değer tahmin eder. Kurallarla çalışır.

Regresyon Modellerinin Mantığı



- Hata payını (Residuals) minimize eden en iyi doğruyu / eğriyi bulma optimizasyonudur.

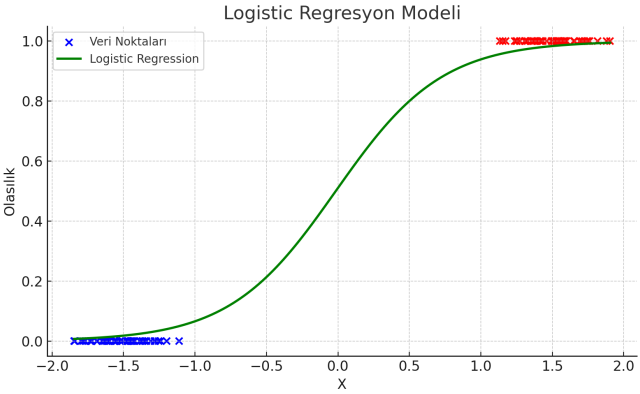
Lojistik Regresyon (Logistic Regression)

- İstatistiksel bir **İkili Sınıflandırma (Binary Classification)** algoritmasıdır.
- Doğrusal denklemin sonucunu **Sigmoid Fonksiyonu** ile 0 ile 1 arasına sıkıştırarak bir Olasılık değerine dönüştürür.

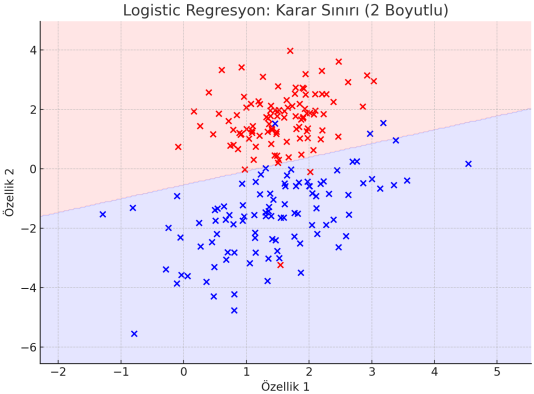
$$\hat{y} = \frac{1}{1 + e^{-(wX+b)}}$$

- **Karar Sınırı (Decision Boundary):** Olasılık 0.5'ten büyükse Sınıf 1, küçükse Sınıf 0 olarak tahmin üretir.
- **Artıları:** Çok hızlıdır, istatistiksel olarak yorumlanması kolaydır.
- **Eksileri:** Veriler doğrusal (lineer) olarak ayrılamıyorsa başarı oranı düşer.

Lojistik Regresyon Karar Sınırı



Lojistik Regresyon (Sigmoid Fonksiyonu)



Lojistik Regresyon

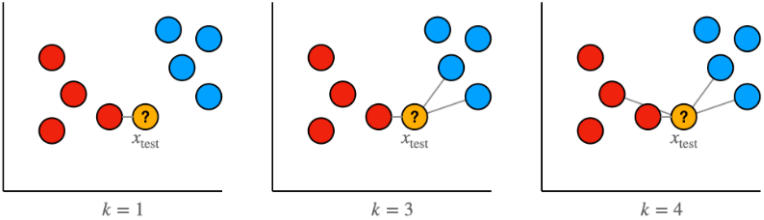
```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

- `max_iter=1000`: Optimizasyon sürecinin (Gradient Descent) adım sayısını belirler.

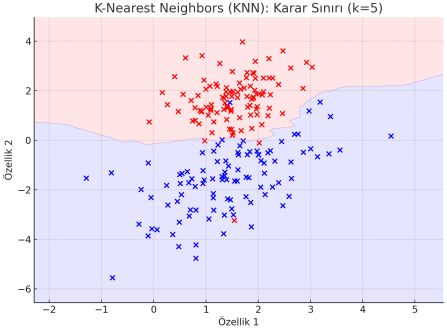
K-En Yakın Komşu (K-Nearest Neighbors - KNN)

- **Lazy Learning (Tembel Öğrenme):** Hesaplama tahmin anında yapılır.
- Yeni gelen verinin uzaydaki konumuna bakıp **Öklid veya Manhattan uzaklığı** ile en yakın K adet komşusunu bulur.
- Sınıflandırma için **Çoğunluk Oylaması (Majority Voting)**, regresyon için komşuların **Ortalaması** alınır.
- **Artıları:** Çok basit ve anlaşılırdır. Non-lineer karmaşık verilerde güçlüdür.
- **Eksileri (Boyut Laneti - Curse of Dimensionality):** Sütun sayısı (boyut) arttıkça uzaklık metrikleri anlamsızlaşır.

K-En Yakın Komşu (KNN) Çalışma Mantığı



KNN’de K Parametresinin Seçimi



- K çok küçükse gürültüye duyarlı olur (Overfitting). K çok büyükse sınıflar birbirine karışır (Underfitting). Genelde **tek sayılar** seçilir.

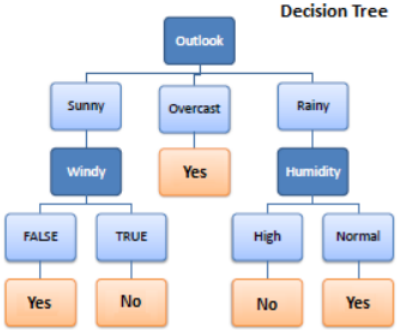
Karar Ağacı (Decision Tree)

- Veri setini en iyi ayıran (Information Gain - Bilgi Kazancı en yüksek olan) özelliklere göre adım adım **if/else kurallarıyla** dallara böler.
- Matematiksel bir denklem kurmaz.
- Eğer kredi skoru < 500 ise ve yaş < 25 ise RED mantığıyla yaprak düğümlere (leaf nodes) kadar iner.
- **Artıları:** Yorumlanabilirliği en yüksek modeldir (White Box). Veri ölçekleme (Scaling) gerektirmez. Kategorik ve sayısal veriyi bir arada işler.
- **Eksileri:** Aşırı öğrenmeye (Overfitting) çok müsaittir. Ağacın derinliği (max_depth) sınırlandırılmazsa (Budama - Pruning yapılmazsa) her bir veri noktası için özel kural uydurur.

Karar Ağaçları Mimarisi

Predictors | Target

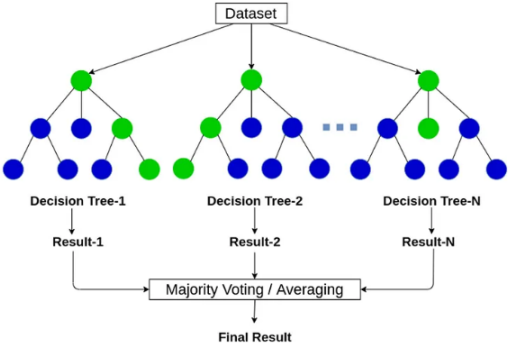
Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No



Topluluk Öğrenmesi: Random Forest (Rastgele Orman)

- **Ensemble Learning (Topluluk Öğrenmesi)**
- Random Forest verinin farklı alt kümelerini (Bagging/Bootstrap) kullanarak yüzlerce Karar Ağacı eğitir. Sınıflandırma için en çok oyu alan sınıfı seçer.
- **Örnek:** Hastalığı tek bir doktora sormak yerine yüzlerce doktordan oluşan bir kurula danışmak.
- **Artıları:** Karar ağaçlarının Overfitting sorununu çözer.
- **Eksileri:** Eğitim süresi uzundur.

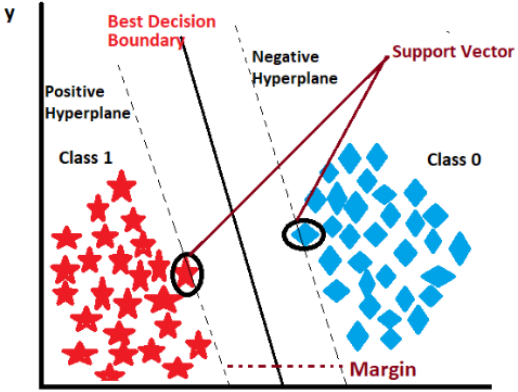
Random Forest Karar Mekanizması



Destek Vektör Makineleri (Support Vector Machines - SVM)

- Sınıfları birbirinden ayıran ideal Hiperdüzlemi (Hyperplane) çizer.
- Amaç aralarındaki boşluğu / marjı (Margin) **maksimize etmektir**. Bu marjı belirleyen sınır noktalara **Destek Vektörü** denir.
- **Kernel Trick:** Veriler 2 boyutlu uzayda düz bir çizgiyle ayrılamıyorsa (Linearly Inseparable) SVM veriyi sanal olarak 3. boyuta katlar ve aralarından düz bir tabaka geçirerek boyutları böler.
- **Artıları:** Yüksek boyutlu verilerde ve karmaşık sınır çizgilerinde güçlüdür.
- **Eksileri:** Ölçekleme (Scaling) zorunludur. Eğitim süresi uzundur ve büyük verilerde yavaşlar.

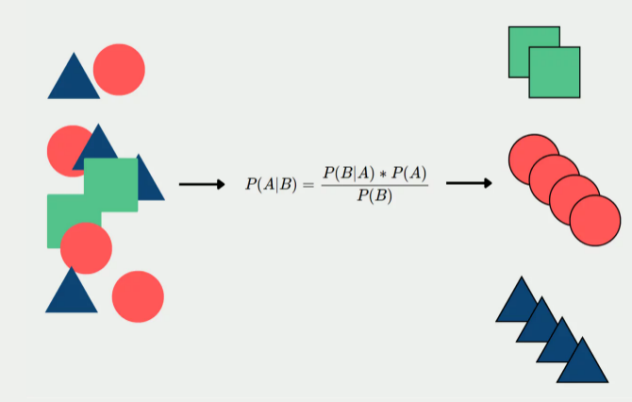
SVM: Maksimum Marj (Margin) Kavramı



Naive Bayes

- **Bayes Teoremi'ni** kullanır. Koşullu olasılık hesaplar: "Elimdeki kelimelere bakarak bu e-postanın SPAM olma ihtimali nedir?
- **Neden Naive (Saf/Basit)?** Tüm özelliklerin birbirinden bağımsız olduğunu varsayar.
- **Kullanım Alanı:** Doğal Dil İşleme (NLP), Duygu Analizi (Sentiment Analysis) ve Metin Sınıflandırma (Text Classification).
- **Artıları:** Hızlı eğitilir. Az veriyle bile başarılıdır.
- **Eksileri:** Bağımsızlık varsayımı gerçek hayatta pek karşılık bulmaz.

Naive Bayes: Bayes Teoremi



Yapay Sinir Ağı (Artificial Neural Networks - ANN)

- **Derin Öğrenmenin (Deep Learning)** temelini oluşturan mimaridir.
- **Yapı:** Girdi Katmanı (Input) → Gizli Katmanlar (Hidden Layers) → Çıktı Katmanı (Output).
- Nöronlar girdileri ağırlıklarla (Weights) çarpar, bias ekler ve **Aktivasyon Fonksiyonundan** (Sigmoid, ReLU) geçirerek bir sonraki katmana iletir.
- Öğrenme **Geri Yayılım (Backpropagation)** algoritmasıyla hataların ağ boyunca geriye doğru dağıtılması ve ağırlıkların güncellenmesiyle gerçekleşir.
- **Artıları:** Görüntü (CNN) ve ses/metin (RNN/Transformer) verilerinde başarılıdır.
- **Eksileri:** İşlem gücü (GPU) ve büyük miktarda miktarda veri ister. Kara kutudur.

Sınıflandırma Metrikleri ve Performans Ölçümü

- **Temel Metrikler**
  - **Accuracy (Doğruluk):** Doğru tahmin sayısı / Toplam tahmin sayısı.
  - **Precision (Kesinlik):** Pozitif dediklerimizin yüzde kaçı gerçekten pozitif?
  - **Recall (Duyarlılık):** Gerçek pozitiflerin yüzde kaçını yakalayabildik?
  - **F1 Skoru:** Precision ve Recall'un dengeli (harmonik) ortalaması.
  - **Confusion Matrix:** Hataların ve doğruların detaylı dökümü.

Karmaşıklık Matrisi (Confusion Matrix)

	Tahmin: Pozitif	**Tahmin: Negatif**
Gerçek: Pozitif	**TP** (Doğru Pozitif)	**FN** (Yanlış Negatif - Tip II Hata)
Gerçek: Negatif	**FP** (Yanlış Pozitif - Tip I Hata)	**TN** (Doğru Negatif)

- **FN (False Negative):** Tıbbi teşhis gibi kritik konularda çok risklidir (Hastaya sağlıklısın demek).
- **FP (False Positive):** Spam filtrelemede kullanıcıyı rahatsız eder (Önemli maili çöpe atmak).

Precision (Kesinlik) ve Recall (Duyarlılık) Arasındaki Fark

Örnek 1 – Spam E-Posta Filtresi (Precision Odaklı)

- Yanlış alarm (FP) istemiyoruz. Birkaç spam mailin (FN) gelen kutusuna düşmesi, önemli bir iş mailinin silinmesinden daha iyidir.
- **Kural:** Spam dediklerim gerçekten spam olmalı.

Örnek 2 – Hastalık Taraması (Recall Odaklı)

- Hastalığı kaçırmak (FN) ölümcüldür. Sağlıklı birine yanlışlıkla hastasın (FP) deyip ek testler yapmak kabul edilebilirdir.
- **Kural:** Hastalığı olan hiç kimseyi kaçırmamalıyım.

F1 Skoru ve Dengesiz Veri (Imbalanced Data)

Örnek – Sahtekarlık Tespiti (Fraud Detection)

- Verilerin %99'u yasal normal işlem, sadece %1'i sahtekarlık olsun.
- Model hiçbir şey öğrenmeden her işleme Normal dese **Accuracy (Doğruluk) = %99** çıkar.
- Ancak sahtekarlığı yakalayamaz → Recall = 0.
- **Çözüm:** Precision ve Recall'un harmonik ortalaması olan **F1 Skoru** kullanılır. Dengesiz senaryolarda genel dengeyi yansıtır.

Precision – Recall – F1 Karşılaştırması

Senaryo	TP	FP	FN	Precision	Recall	F1 Skoru
A – Dengeli Model	80	20	20	0.80	0.80	**0.80**
B – Yüksek FP (Yanlış Alarm)	80	40	20	0.67	0.80	**0.73**
C – Yüksek FN (Kaçırma)	80	20	40	0.80	0.67	**0.73**

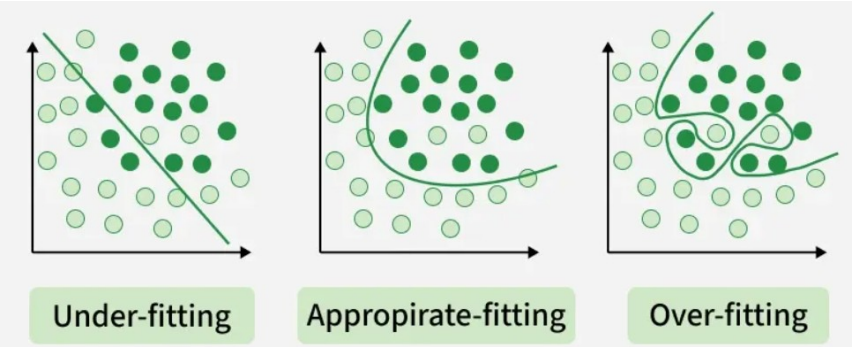
Sınıflandırma Metriklerinin Hesaplanması

```
from sklearn.metrics import accuracy_score, precision_score,
    recall_score, f1_score
from sklearn.metrics import confusion_matrix,
    classification_report

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))

print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test
    , y_pred))
```

Aşırı Öğrenme (Overfitting) ve Yetersiz Öğrenme (Underfitting)

- **Underfitting (Yetersiz Öğrenme)**
  - Eğitim verisindeki temel örüntüleri yakalanmaz. Hem eğitim hem test başarısı çok düşüktür.
- **Overfitting (Aşırı Öğrenme)**
  - Model eğitim verisini ezberler.
- **Amaç:** Modelin ezberlemediği ama mantığı öğrendiği o **dengeli (Optimum Fit)** noktayı bulmaktır.

Aşırı Öğrenme (Overfitting) ve Yetersiz Öğrenme (Underfitting)



Model Performansını İyileştirme

- **Başarısızlığın (Overfitting/Underfitting) Nedenleri**
  - Yanlış model seçimi (Karmaşık veriye çok basit doğrusal model kurmak).
  - Eksik veya önemsiz özellikler (Feature Selection eksikliği).
  - Yetersiz veri miktarı veya dengesiz sınıflar.
- **İyileştirme (Tuning) Yolları**
  - Daha fazla ve çeşitli veri toplamak.
  - Hiperparametreleri optimize etmek (Grid Search).
  - Veri temsiliyetini değiştirmek (Özellik Mühendisliği).

Veri Temsilinin (Representation) Etkisi

- **Veri Temsili:** Modelin matematiksel olarak anlayacağı formattır.
- **Yaş:** `age = 42` (Doğru) vs `age = "Orta"`.
- **Cinsiyet:** `Erkek = 1, Kadın = 2`. One-Hot Encoding.
- **Tarih:** `Gun = 7` (Pazar) yerine `Tatil = True/False` şeklinde yeni bir özellik eklemek.

Makine Öğrenmesinde Döngüsel Süreç

- Modelleme sürekli tekrarlanan bir mühendislik döngüsüdür.

- **1. Veri:** Hazırlanır, temizlenir, dönüştürülür.
- **2. Öğrenme:** Model eğitilir (`fit`).
- **3. Tahmin:** Test verisine uygulanır (`predict`).
- **4. Değerlendirme:** Metriklerle ölçülür (`accuracy, f1, confusion matrix`).
- **5. Geri Dönüş:** İyileştirme için tekrar Veri'ye veya Model mimarisine dönülür.

Yapay zeka modelleri verilerle yetiştirilir.

Kümeleme (Clustering) Nedir?

- Etiketlenmemiş (hedef değişkeni olmayan) verileri benzerliklerine göre gruplara ayırma yöntemidir.
- Gözetimsiz öğrenme (Unsupervised Learning) türüdür.
- **Hedef:** Aynı kümedeki verilerin birbirine olabildiğince benzemesi, farklı kümelerdeki verilerin birbirinden olabildiğince farklı olmasıdır.

Örnek Uygulamalar

- Müşteri segmentasyonu (Harcama alışkanlıklarına göre kitleleri gruplama)
- Haberleri veya dokümanları konu başlıklarına göre otomatik gruplama

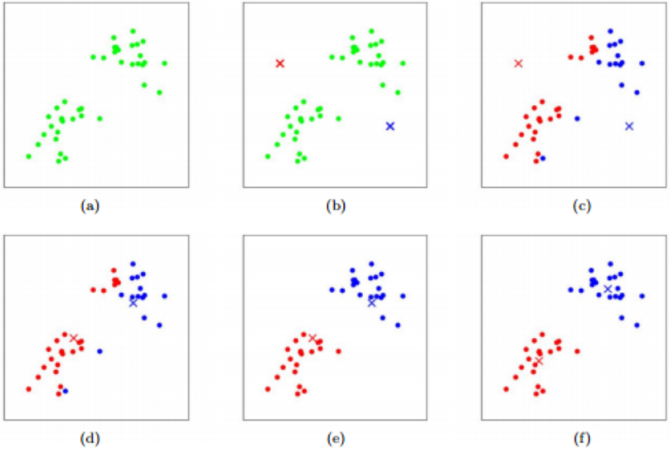
Algoritmalar: K-Means, DBSCAN, Hiyerarşik Kümeleme (Hierarchical Clustering)

K-Means Kümeleme (K-Ortalamalar)

- Belirli sayıda merkez (K - Centroid) rastgele uzaya yerleştirilir.
- Her veri noktası en yakın merkeze atanarak kümeler oluşturulur.
- Merkezler içindeki noktaların ortalaması alınarak yeni konumlarına güncellenir.
- İşlem merkezler sabit kalana (convergence) dek tekrarlanır.

Artıları: Hızlıdır, mantığı sezgiseldir ve büyük verilerde iyi çalışır.
Eksileri: Küme sayısı (K) önceden belirlenmelidir. Aykırı değerlere hassastır.

K-Means Çalışma Mantığı



DBSCAN (Density-Based Clustering)

- Yoğunluk tabanlı kümeleme: **Yakın ve kalabalık** noktalar aynı kümeye girer.
- Seyrek bölgelerdeki yalnız noktalar **Gürültü / Dışta (Outlier)** sayılır.

Örnek: GPS konumlarında yoğun toplanma alanlarını (ziyaret edilen yerler) bulma.

Artıları: Küme sayısı vermek gerekmez. Düzensiz şekillerde ve gürültülü veride çok etkilidir.
Eksileri: Farklı yoğunluklu kümeler bir arada olursa problem olabilir.

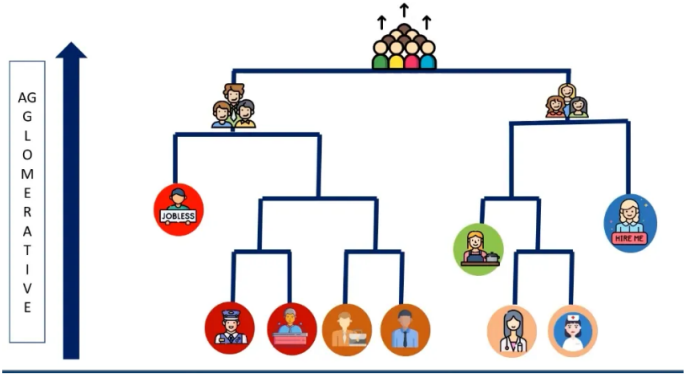
Hiyerarşik Kümeleme (Hierarchical Clustering)

- **Kümeler arası benzerlik** ölçülerek küçük gruplar adım adım birleştirilir (Agglomerative - Aşağıdan Yukarıya).
- Veriler bir ağaca (Dendrogram) dönüşür.
- İstenilen küme sayısı: Ağaç yapısı belirli bir mesafeden **kırpılarak** elde edilir.

Örnek: Genetik benzerliğe göre canlıları sınıflandırmak (soy ağacı benzeri yapı).

Artıları: Küme sayısını baştan belirtmeye gerek yoktur. Dendrogram ile sezgisel analiz yapılabilir.
Eksileri: Büyük veri setlerinde **hesaplama maliyeti yüksektir**.

Hiyerarşik Kümeleme Dendrogramı



Uygulama: Diabetes (Diyabet) Veri Seti

- **Kaynak:** `sklearn.datasets.load_diabetes()`
- **Örnek sayısı:** 442 hasta
- **Özellik sayısı:** 10 sayısal özellik
- **Hedef (target):** Hastanın 1 yıl sonraki hastalık ilerleme ölçüsü (sürekli bir değer).
- **Amaç:** Hastaya ait ölçümlere bakarak hastalığın ne kadar ilerleyeceğini matematiksel olarak tahmin etmek.

- bmi: Vücut kitle indeksi
- bp: Kan basıncı
- s1-s6: Çeşitli kan testleri sonuçları

Regresyon problemi — Bir sayısal değer tahmin edilecek.

Veri Setini Yükleme

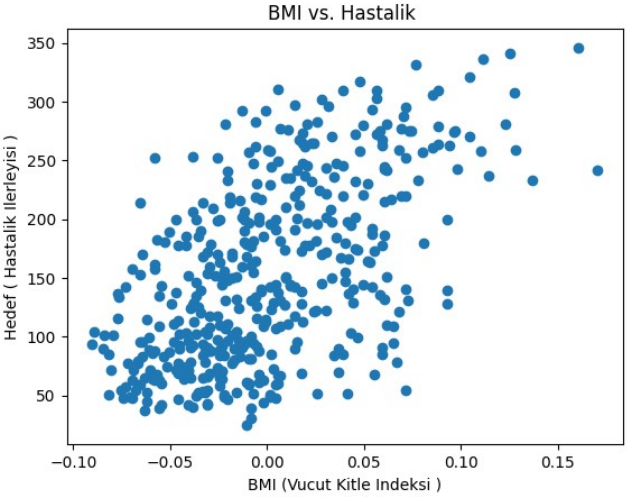
```python
from sklearn.datasets import load_diabetes

diabetes = load_diabetes()
X = diabetes.data
y = diabetes.target

print(X.shape)  # Cikti: (442, 10)
print(diabetes.feature_names)
```

Özellik Hedef İlişkisi

```
import matplotlib.pyplot as plt

# 2. indeks olan BMI sutununu aliyoruz
plt.scatter(X[:, 2], y)
plt.xlabel("BMI (Vucut Kitle Indeksi)")
plt.ylabel("Hedef (Hastalik Ilerleyisi)")
plt.title("BMI vs. Hastalik")
plt.show()
```

- Bazı özelliklerle hedef değişken arasında görsel olarak doğrusal bir ilişki (korelasyon) gözlenebilir.

Özellik Hedef



Regresyon Modeli Eğitimi

```python
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Egitim (%80) ve Test (%20) bolunmesi
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Modeli olustur ve egit
model = LinearRegression()
model.fit(X_train, y_train)
```

Model Başarısının Hesaplanması

```
from sklearn.metrics import mean_squared_error,
    mean_absolute_error, r2_score

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("R^2 Skoru:", r2)
```

Regresyon Değerlendirme Ölçütleri

- **Mean Squared Error (MSE):** Tahmin hatalarının karesinin ortalaması.
  - Hataları büyütür, uç değerlere (outliers) karşı duyarlıdır.
- **Mean Absolute Error (MAE):** Hataların mutlak değerlerinin ortalaması.
  - Yorumu kolaydır (”Ortalama X birim hata yapıyoruz”), uç değerlere daha az duyarlıdır.
- **R^2 Skoru (Determinasyon Katsayısı):** Modelin açıklayabildiği değişkenlik oranı (0–1 arası).
  - 1’e yaklaştıkça model daha başarılıdır.

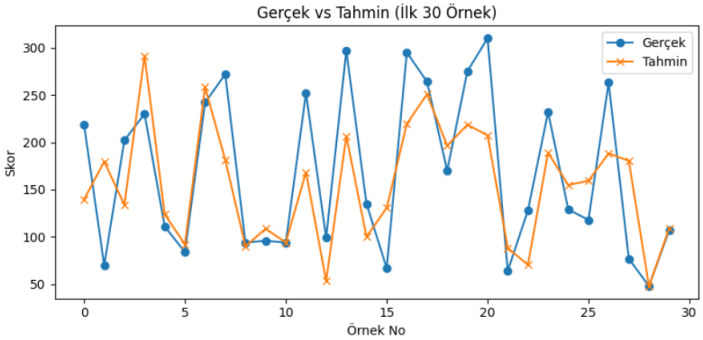
Not: Küçük MSE ve MAE → daha iyi tahmin, Yüksek R^2 → güçlü ilişki.

Gerçek ve Tahmin Karşılaştırması

```
plt.plot(y_test, label="Gercek")
plt.plot(y_pred, label="Tahmin")
plt.legend()
plt.title("Gercek vs. Tahmin")
plt.show()
```

- Gerçek değer ile tahmin arasındaki fark görsel olarak incelenir. Çizgiler birbirine ne kadar yakınsa model o kadar başarılıdır.

Sonuç Grafiği



Iris Veri Seti ile Kümeleme

```python
from sklearn.datasets import load_iris

iris = load_iris()
X = iris.data
y_true = iris.target
feature_names = iris.feature_names
```

- 3 sınıflı (farklı çiçek türü), 4 özellikli etiketlenmiş veri.
- `X`: Kümeleme algoritmalarına giriş olarak verilecek.
- `y_true`: son aşamada değerlendirme için kullanılacak.

KMeans ile Kümeleme

```
from sklearn.cluster import KMeans

# n_clusters=3 cunku 3 cicek turu oldugunu biliyoruz
kmeans = KMeans(n_clusters=3, random_state=0)
labels_kmeans = kmeans.fit_predict(X)
```

- Küme sayısı kullanıcı tarafından belirlenir (`n_clusters=3`).
- Küme merkezleri iteratif olarak güncellenir.
- Sonuç: Her örnek için bir küme etiketi (`labels_kmeans`) üretilir.

DBSCAN ile Kümeleme

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.6, min_samples=4)
labels_dbscan = dbscan.fit_predict(X)
```

- Yoğunluk temelli: Küme sayısı önceden verilmez.
- `eps` (yarıçap) ve `min_samples` (minimum nokta) parametreleri kritik rol oynar.

Hiyerarşik Kümeleme (Agglomerative)

```
from sklearn.cluster import AgglomerativeClustering

agg = AgglomerativeClustering(n_clusters=3)
labels_agg = agg.fit_predict(X)
```

- Küçük kümeleri birleştirerek (mesafe ölçümüyle) yukarıdan aşağıya ilerler.
- Çiçeklerin 3 sınıfa ayrılmasını istediğimiz için ağacı 3 küme kalacak şekilde keseriz (`n_clusters=3`).

PCA ile Görselleştirmeye Hazırlık

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
```

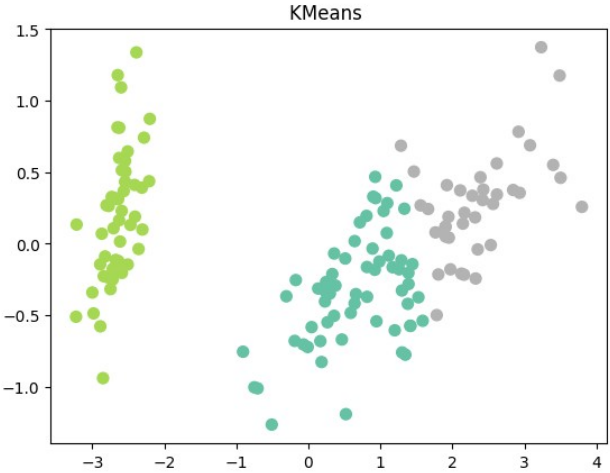
- PCA gözetimsiz bir boyut indirgeme algoritmasıdır.
- Kümeleme sonuçlarını görsel olarak 2D düzlemde karşılaştırmaktır.

Kümeleme Sonuçlarının Görselleştirilmesi

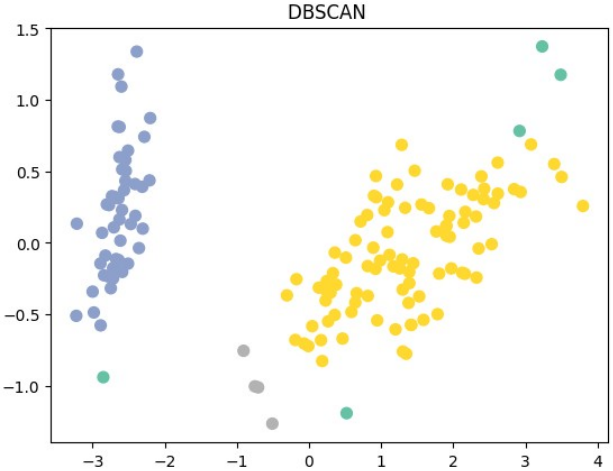
```
def plot_clusters(title, labels):
    plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap="Set2",
        s=50)
    plt.title(title)
    plt.show()

plot_clusters("KMeans", labels_kmeans)
plot_clusters("DBSCAN", labels_dbscan)
plot_clusters("Agglomerative", labels_agg)
```

- PCA bileşenleri (2D) üzerinde kümeler çizilir.
- Renkler (`c=labels`) algoritmaların tahmin ettiği kümelere göre atanır.

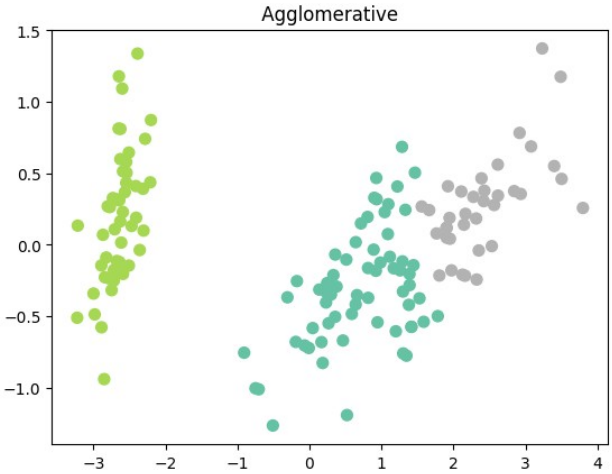
Sonuçlar



Sonuçlar



Sonuçlar



Yorum ve Uygunluk

- **KMeans:** Küme sayısı biliniyorsa başarılı.
- **DBSCAN:** Gürültülü, dağınık ve iç içe geçmiş verilerde avantajlı.
- **Agglomerative:** Küme yapısı belirsizken dendrogram ile ağaç yapısını görmek istendiğinde iyi çalışır.
- **Sonuç:** Her kümeleme algoritması veri yapısına göre özel olarak seçilmelidir.

Kümeleme Sonuçlarını Etiketlerle Kıyaslamak

- Iris veri seti sınıflandırma amaçlıdır.
- Kümeleme algoritmaları gözetimsiz (unsupervised) olduğu için çıktıları gerçek etiketlerin rakamlarıyla **doğrudan eşleşmeyebilir:**
  - Gerçek etiket (Sınıflar): `[0, 1, 2, ...]`
  - Kümeleme çıktısı: `[2, 0, 1, ...]` olabilir.
- Model hata mı yaptı?

Veri Seti: Breast Cancer

- **Amaç:** Hastadan alınan hücre özelliklerine bakarak tümörün iyi huylu (benign) ya da kötü huylu (malignant) olup olmadığını tahmin etmek.
- **Özellik sayısı:** 30 (Örnek: hücre yarıçapı, dokusu, çevresi vs.)
- **Sınıf sayısı:** 2 (Benign = İyi Huylu, Malignant = Kötü Huylu)
- **Kullanılan kütüphane:** `sklearn.datasets.load_breast_cancer`

Kullanılan Sınıflandırma Algoritmaları

- **Logistic Regression:** Basit, doğrusal olasılık modeli.
- **Karar Ağacı (Decision Tree):** İnsan mantığına yakın, if/else kural ayrımı.
- **Random Forest:** Birden fazla ağacın ortak kararı (Topluluk Öğrenmesi).
- **Support Vector Machine (SVM):** Sınıfları ayıran maksimum marjlı hiperdüzlem.
- **K-Nearest Neighbors (KNN):** En yakın komşulukların oylamasına göre karar.

Veriler eğitim/test olarak bölünecek ve model performansları için `StandardScaler` ile ölçeklenecektir.

Kütüphanelerin Yüklenmesi

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix
```

Sınıflandırma Algoritmaları

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
```

Veri Seti Hazırlığı ve Veri Sızıntısı (Data Leakage) Önlemi

```
data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

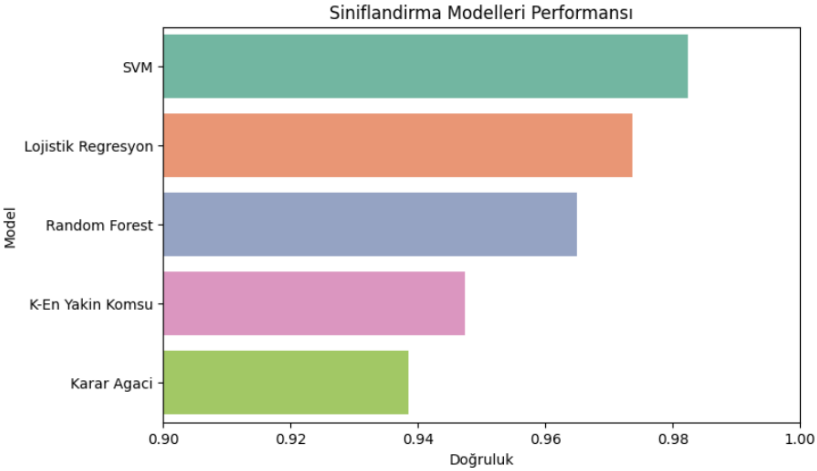
Modellerin Tanımlanması

```
models = {
    "Lojistik Regresyon": LogisticRegression(max_iter=1000),
    "Karar Agaci": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "SVM": SVC(),
    "K-En Yakin Komsu": KNeighborsClassifier()
}
```

Model Eğitimi ve Değerlendirme

```
results = []

for name, model in models.items():

    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    results.append((name, acc))
```

Değerlendirme

```
results_df = pd.DataFrame(results, columns=["Model", "Dogruluk
    Skoru"])
results_df = results_df.sort_values(by="Dogruluk Skoru",
    ascending=False)

print(results_df)

plt.figure(figsize=(8,5))
sns.barplot(x="Dogruluk Skoru", y="Model", data=results_df,
    palette="Set2")
plt.title("Siniflandirma Modelleri Performans Karsilastirmasi")
plt.xlabel("Dogruluk Skoru (Accuracy)")
plt.xlim(0.9, 1.0)
plt.show()
```

Sonuç Raporu